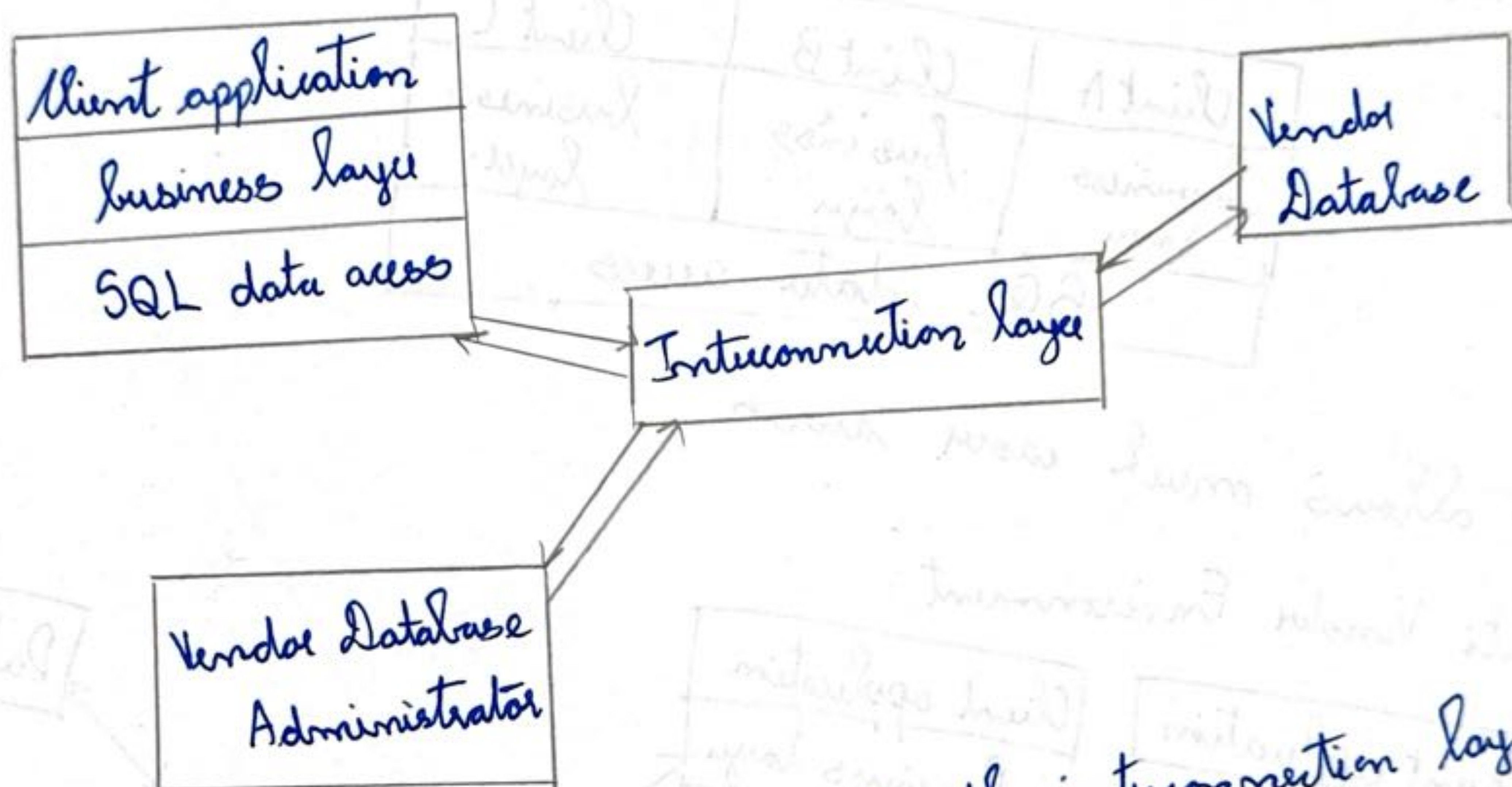


1. (c) SQL/content-based middleware are those middleware architecture that allows you to filter values / apply conditions based on the content inside them.

SQL/content-based middleware can be implemented in single vendor where there is only one vendor or multi-vendor when multiple different vendors access the same database.

Single Vendor Environment

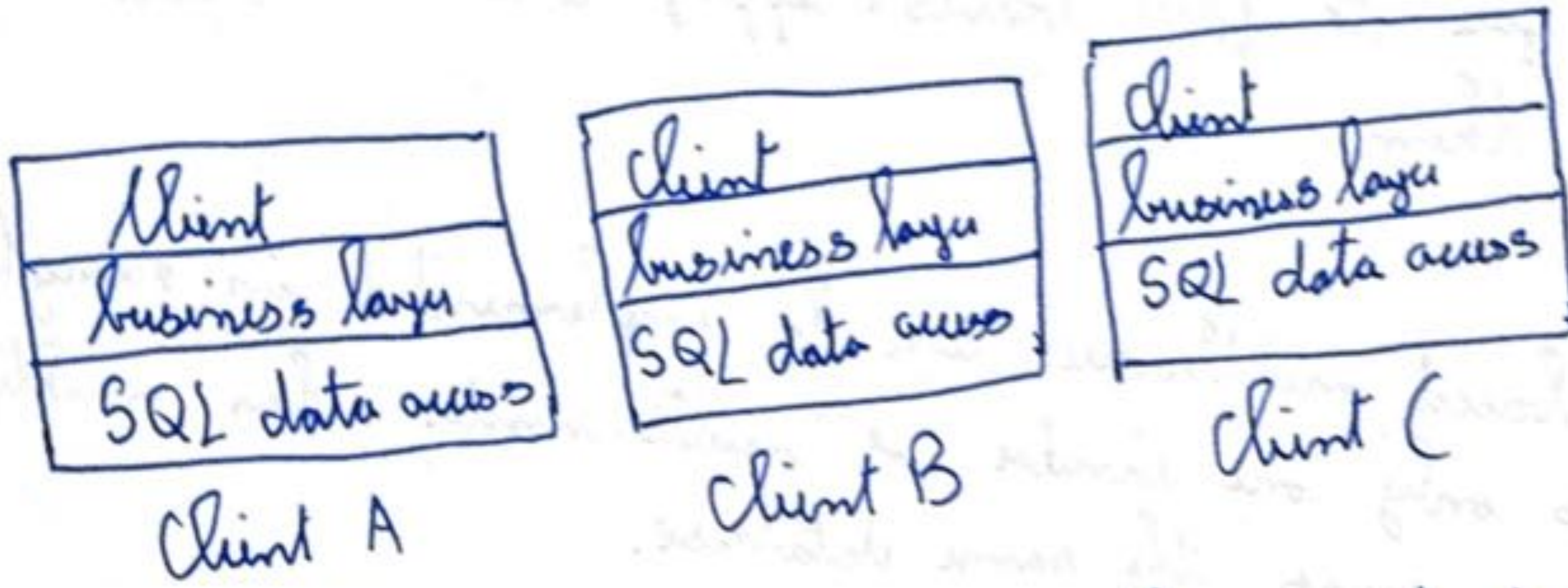


The client application sends the requests to the interconnection layer, the sql requests are then validated by the database administrator, this ensures that no critical information is leaked as well as that the client has permissions to access the data.

Once the sql request is validated it is sent to the vendor database and the result is returned to the client.

The client processes this result and displays the final output to the user.

There is a chance for each client to have separate data access layer

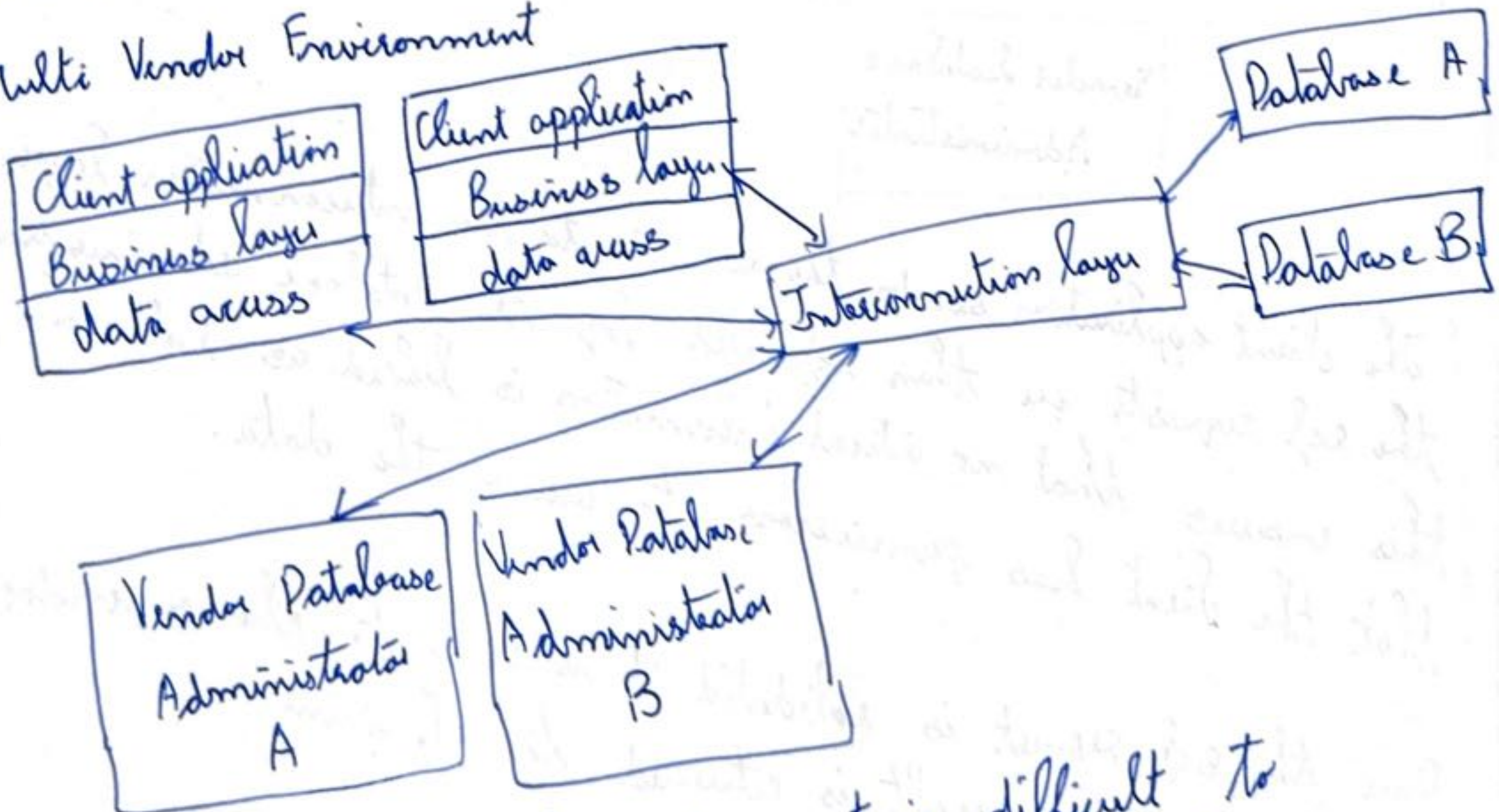


This is too cumbersome to manage hence it is better for multiple clients to have same SQL data access layer.



This allows much easier access.

Multi Vendor Environment



Since there are multiple vendors, it is difficult to have a middleware that supports the protocols of different vendors.

To solve this we replace the Interconnection Layer with

Formats and Protocol (FAP)

This allows multiple different vendors to use a common standard format, making SQL data access much easier.

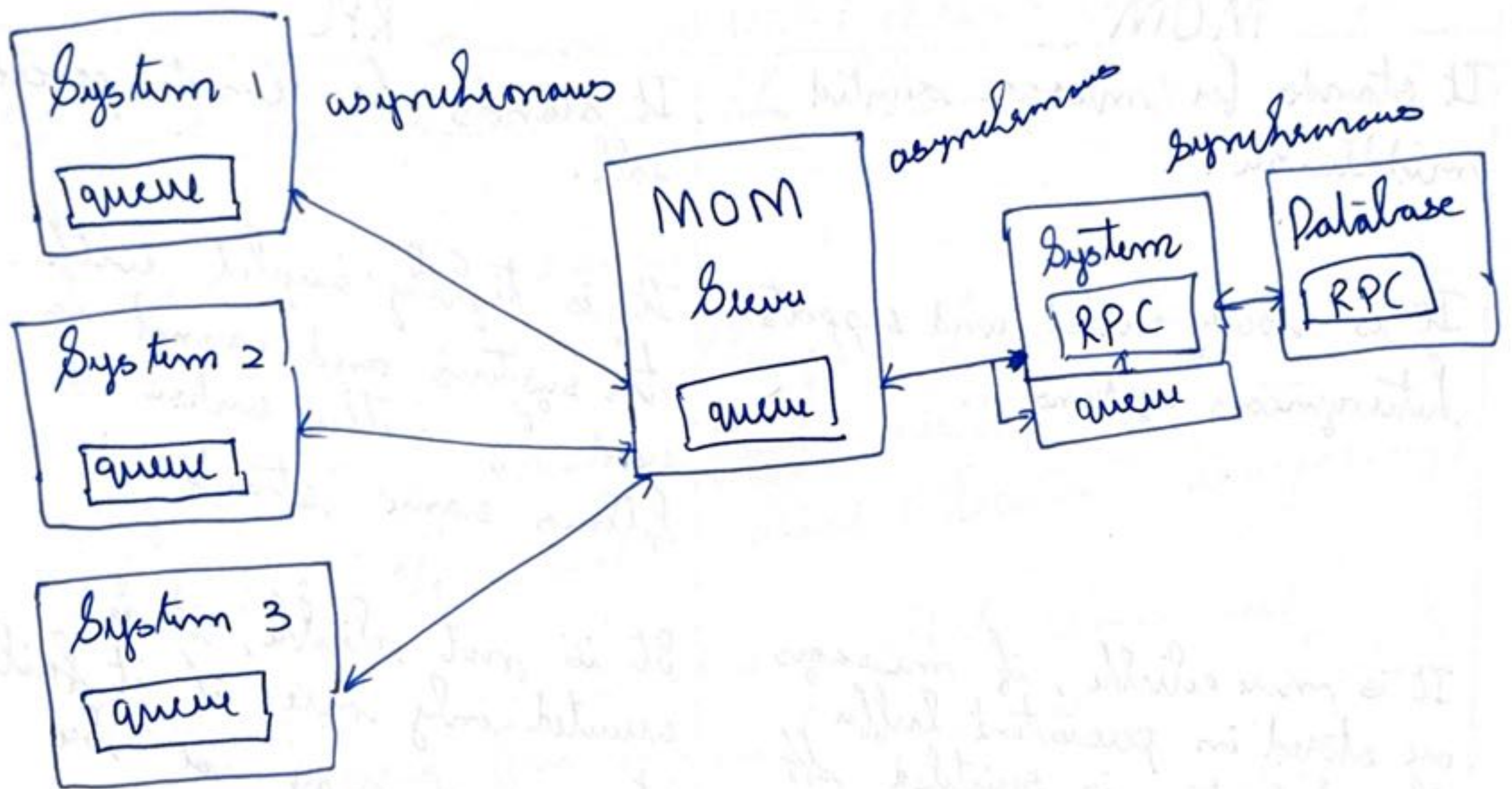
(i) The middleware to be used is RPC integrated with MOM.

Since there are various different types of heterogeneous systems, it is not possible to implement a common RPC since they follow different standards and since many different systems connect to a central server, RPC is synchronous and this might cause delay.

Hence the different systems send their request via MOM (Message Oriented Middleware) which stores them in a queue as the requests come. It follows FIFO principle and reads the request based on arrival.

The MOM server then sends the request to a system which converts it into a common format and uses RPC to retrieve the data. This simplifies the process of data retrieval, it is okay if it is synchronous since only one SQL request made at a time.

The MOM queues the result in its buffer and sends them to the respective client one by one.



4

(ii)

MDM

It stands for message oriented middleware.

It is loosely coupled and supports heterogeneous systems.

It is more reliable, if messages are stored in persistent buffer then if system is switched off data is not lost.

It is highly scalable since it is asynchronous, it can support multiple different systems sending requests at the same time.

It uses a queue as buffer.

It does not mask the request.



RPC

It stands for remote procedure call.

It is tightly coupled with the system and cannot be used by another unless it follows same standard.

It is not reliable, it is executed only once if it fails it may or may not be retried again.

It is not scalable since it is synchronous, the next request can only be made when the result of previous request is returned.

It does not have any buffer.

It masks the request, user sees it like a local function call, does not know where it is executed.

(c) The internet can act as a NOS (Network Operating System) for cloud based services in the following ways:

- It provides directory services

It is possible to browse through the internet and get any content you want without needing to know which server the content is stored (location transparency)

Same content can be stored on multiple servers and can be accessed from anywhere (Idempotency and replication)

It is also possible to delete files on different machines using VDI or remote desktop.

- It provides Time and clock synchronization.

The internet allows different systems to correct their internal times with actual time.

It allows different systems to connect to genuine sources of time like satellite and radio clocks.

- It provides certain degree of authentication and security

HTTP and S-HTTP ensure that the content shared across the internet is uniform and follow a certain degree of standardization and is secure

Protocols such as SSL (Secure Sockets Layer) and TLS ensure that two entities are verified and authenticated before any information can be shared between them.



It acts as a middleware.
API calls to server can be made using the internet. The user need not know the internal details. He can call the API like a normal function (RPC). The API uses the internet to retrieve the required data.

It allows you to communicate with other systems as well as transfer files.

The internet uses various protocols like SMTP (Simple Mail Transfer Protocol), that allows you to send messages as well as transfer files between other users.

It allows asynchronous communication since messages are buffered on a mail server before the client is ready to receive them.

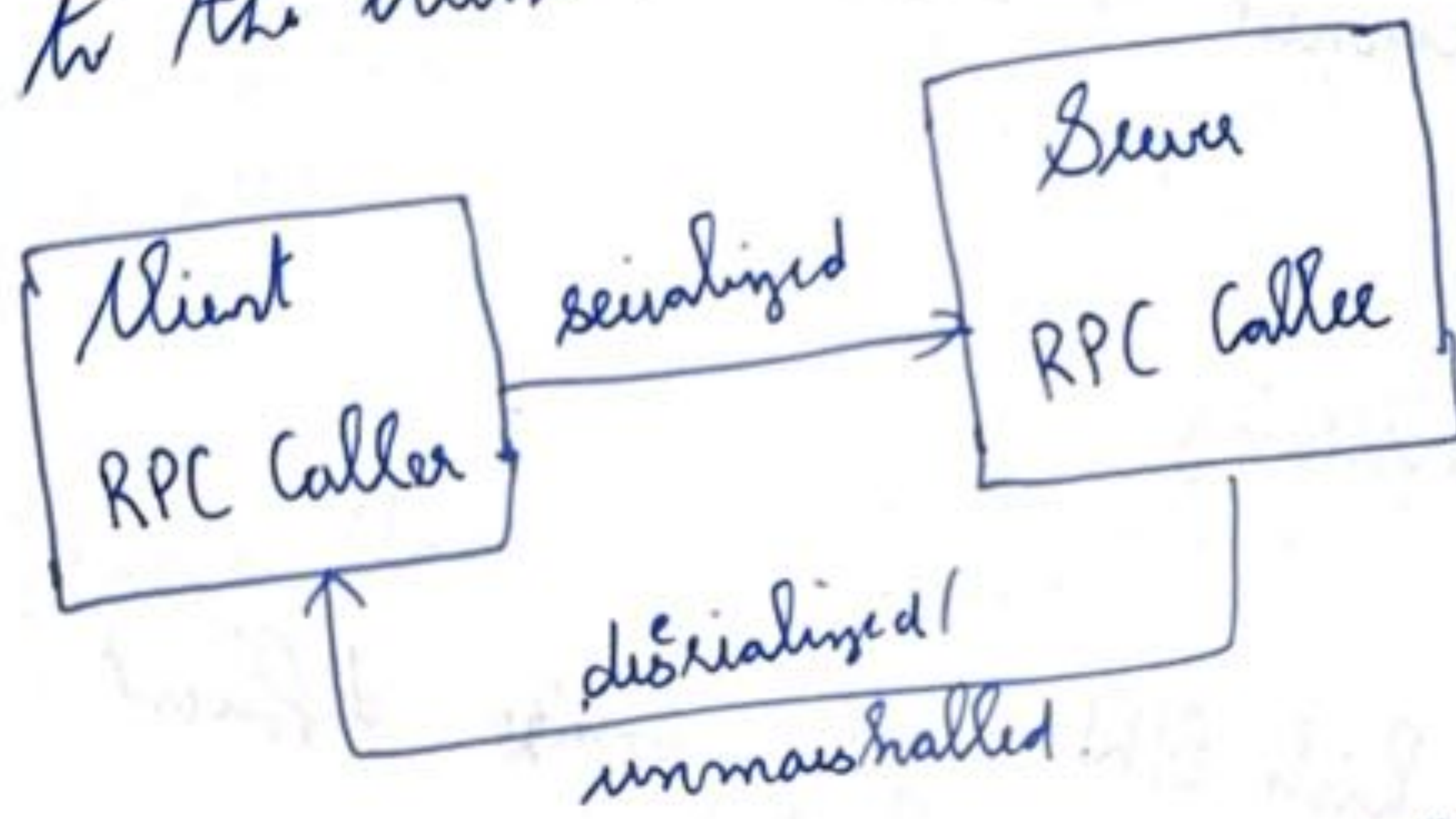
It provides naming services.

The location of a object on the internet is never revealed. it is kept hidden.

8

b
(i) DCE can be applied in the following ways

RPC stands for remote procedure call, the user calls a function locally as he normally does, the function is ~~is~~ serialized and sent to the server. The server unserializes it, executes it and returns the answer to the client. The execution of the function is blackboxed to the client.



In distributed naming services, the name of the content/resource only contains its name. It does not contain any other details. If a user wants to access a particular contents he only needs to know its name, he does not need any other details like where it is stored (Location transparency). This is provided by DCE.

4 In distributed security services, by the use of SSL and TLS communication between two systems in a DCE is secured. DCE also provides other security feature like Kerberos which provides client with TGT (Ticket Granting Ticket) which it can then use to generate a ST (Session Ticket).

b
(ii)

Thin Client computing is widely used in environments where the client can be a minimal device like handheld scanner, teleinformation, simple sensors etc and does not process heavy information.

Hardware needs of Client are

- minimal CPU/GPU
- Minimum Storage (to run boot s/w)
- Network Connectivity (to connect to server)

HW needs of server are

- Heavy GPU/CPU for processing
- Large Storage
- Network Connectivity with high BW (to manage different clients)

SW needs of client: Minimal Software, GUI alone

~~HW needs of client:~~

SW needs of server: High complex software

Thin Client computing is a type of computing in which the client does minimal processing and most of the work is outsourced to the server which returns only the final result to the client. For displaying it to the user.

The use scenarios are

- | | | |
|--|------------------------------------|----------------------|
| • Inventory management,
Food Order services | Handheld
device | 0% Local 100% Server |
| • Telemarketing,
marketing | Simple (PU)
screens | 0% Local 100% Server |
| • Knowledge
workers | Financial
Management | 50% Local 50% Server |
| • Power Users | Complex scientific
computations | 100% Local 0% Server |

Main Challenge is network latency and when you cannot connect to server. Not suitable when heavy processing is to be done.

